

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Заверюха Т.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 27.09.26

Москва, 2026

Постановка задачи

Вариант 7.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в pipe1. Родительский процесс читает из pipe1 и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

В файле записаны команды вида: «число число число». Дочерний процесс считает их сумму и выводит результат в стандартный поток вывода. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void);` – создает дочерний процесс.
- `int pipe(int *fd);` – создание канала для передачи данных
- `int execv(const char *pathname, char *const argv[])`
- `pid_t wait(int *status)` – ожидает завершения любого дочернего процесса
- `int open(const char *pathname, int flags)` – открывает файл по указанному пути с соответствующим флагом
- `int close(int fd)` – закрывает файл
- `int dup2(int oldfd, int newfd)` – дублирует файловый дескриптор

В рамках данной лабораторной работы я написал программу, которая в ходе своей работы создает дочерний процесс, переопределяет стандартный поток ввода дочернего процесса открытым файлом, читает из pipe и выводит прочитанное в стандартный поток вывода. Дочерний процесс читает команды из стандартного потока вида «число число число»`endl`, выводит их сумму в стандартный поток вывода, которые перенаправляется в pipe

Код программы

posix_ipc-example-client.c

```
#include <stdint.h>
#include <stdbool.h>

#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
// #include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <fcntl.h>

static char SERVER_PROGRAM_NAME[] = "posix_ipc-example-server";

int main(int argc, char **argv) {
    // Get name of file
    char filename[1024];
    {
        const char prompt[] = "enter filename: ";
        write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);

        ssize_t n = read(STDIN_FILENO, filename, sizeof(filename) - 1);
        if (n <= 0) {
            const char msg[] = "error: failed to read filename\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            exit(EXIT_FAILURE);
        }
        filename[n] = '\0';
        char *nl = strchr(filename, '\n');
        if (nl) *nl = '\0';
    }

    // Open file
    int fd = open(filename, O_RDONLY);
    if (fd == -1) {
        const char msg[] = "error: failed to open file\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }

    // Way to server
    char prospath[1024];
    {
        ssize_t len = readlink("/proc/self/exe", prospath, sizeof(prospath) - 1);
        if (len == -1) { /* Error*/ exit(EXIT_FAILURE); }
        while (prospath[len] != '/') --len;
        prospath[len] = '\0';
    }

    // Open pipe
    int pipe1[2];
```

```

if (pipe(pipe1) == -1) { /* Error */ exit(EXIT_FAILURE); }

// Spawn a new process
pid_t child = fork();
if (child == -1) { /* Error*/ exit(EXIT_FAILURE); }

if (child == 0) {
    close(pipe1[0]);

    dup2(fd, STDIN_FILENO);
    close(fd);

    dup2(pipe1[1], STDOUT_FILENO);
    close(pipe1[1]);

    char path[1024];
    snprintf(path, sizeof(path), "%s/%s", progpath, SERVER_PROGRAM_NAME);
    char *const args[] = {SERVER_PROGRAM_NAME, NULL};
    execv(path, args);
    exit(EXIT_FAILURE);
}

close(pipe1[1]);
close(fd);

char buf[4096];
ssize_t bytes;
while ((bytes = read(pipe1[0], buf, sizeof(buf))) > 0) {
    write(STDOUT_FILENO, buf, bytes);
}

close(pipe1[0]);
wait(NULL);
return EXIT_SUCCESS;
}

```

posix_ipc-example-server.c

```

#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>
// #include <stdio.h>
#include <ctype.h>

static int frmt_int(char *buf, int v) {
    char tmp[12];
    int i = 0;
    int j = 0;
    int u;

    if (v < 0) {
        buf[j] = '-';
        j = j + 1;
    }
}

```

```

        u = -v;
    } else {
        u = v;
    }

    if (u == 0) {
        tmp[i] = '0';
        i = i + 1;
    }

    while (u > 0) {
        tmp[i] = '0' + (u % 10);
        i = i + 1;
        u = u / 10;
    }

    while (i > 0) {
        i = i - 1;
        buf[j] = tmp[i];
        j = j + 1;
    }

    return j;
}

static int frmt_msg(char *buf, float sum, int count) {
    int n = 0;
    const char *p;

    p = "sum = ";
    while (*p != '\0') {
        buf[n] = *p;
        n = n + 1;
        p = p + 1;
    }

    if (sum < 0) {
        buf[n] = '-';
        n = n + 1;
        sum = -sum;
    }

    long sc = (long)(sum * 100.0f + 0.5f);

    n = n + frmt_int(buf + n, (int)(sc / 100));

    buf[n] = '.';
    n = n + 1;

    buf[n] = '0' + (int)((sc / 10) % 10);
    n = n + 1;
    buf[n] = '0' + (int)(sc % 10);
    n = n + 1;

```

```

p = " (numbers: ";
while (*p != '\0') {
    buf[n] = *p;
    n = n + 1;
    p = p + 1;
}

n = n + frmt_int(buf + n, count);

buf[n] = ')';
n = n + 1;
buf[n] = '\n';
n = n + 1;
buf[n] = '\0';

return n;
}

static void process_line(const char *line) {
    float sum = 0.0f;
    int count = 0;
    const char *p = line;

    while (*p != '\0') {
        while (*p && (isspace((unsigned char)*p) || *p == ','))
            ++p;
        if (*p == '\0') break;

        char *endptr = NULL;
        float value = strtod(p, &endptr);
        if (endptr == p) { ++p; continue; }

        sum += value;
        ++count;
        p = endptr;
    }

    char msg[128];
    int32_t len = frmt_msg(msg, sum, count);
    write(STDOUT_FILENO, msg, len);
}

int main(void) {
    char buf[4096];
    ssize_t bytes;

    char line[4096];
    size_t linelen = 0;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        for (ssize_t i = 0; i < bytes; i++) {
            char c = buf[i];

            if (c == '\n') {

```

```

        line[linelen] = '\0';
        process_line(line);
        linelen = 0;
    } else {
        if (linelen < sizeof(line) - 1) {
            line[linelen++] = c;
        }
    }
}

if (bytes < 0) {
    const char msg[] = "error: failed to read from stdin\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    exit(EXIT_FAILURE);
}

if (linelen > 0) {
    line[linelen] = '\0';
    process_line(line);
}

return EXIT_SUCCESS;
}

```

Протокол работы программы

barmalai@Tisha-PC:~/3sem/OS/1lab\$./posix_ipc-example-client filename

enter filename: filename

sum = 7.00 (numbers: 3)

sum = 60.75 (numbers: 3)

sum = -4.00 (numbers: 2)

sum = 0.00 (numbers: 5)

sum = 100.00 (numbers: 1)

sum = 3.01 (numbers: 3)

Вывод

В ходе лабораторной работы я приобрел практические навыки по управлению с процессами в ОС и их взаимодействию с помощью каналов в системе Linux.